

Pearls AQI Predictor

A Serverless Air Quality Index Forecasting System for Lahore

Areeb Zahra · DataScience

1. Project Overview

This project implements an end-to-end, 100% serverless system to forecast the Air Quality Index (AQI) for Lahore, Pakistan, up to 3 days in advance. Rather than training a single model to predict 72 hours ahead directly, the forecast is structured as three separate models, each specialized for a different horizon ($t+24h$, $t+48h$, $t+72h$), since accuracy and the relevant feature relationships change meaningfully with the length of the forecast window.

The system covers the full ML lifecycle: automated data collection, feature engineering and storage, model training and evaluation, model registry, automated retraining and inference via CI/CD, and a public-facing interactive dashboard.

2. Data Sources

- Pollutant data (CO, NO, NO₂, O₃, SO₂, PM_{2.5}, PM₁₀, NH₃): OpenWeather Air Pollution API — current and historical endpoints
- Weather data (temperature, humidity, wind speed, pressure, precipitation): Open-Meteo API (free, no API key required)
- AQI ground truth: computed from PM_{2.5}/PM₁₀ concentrations using the standard EPA AQI breakpoint formula

3. Exploratory Data Analysis

Before feature engineering, the raw pollutant and weather time series were inspected for trends, seasonality, and correlations with AQI.

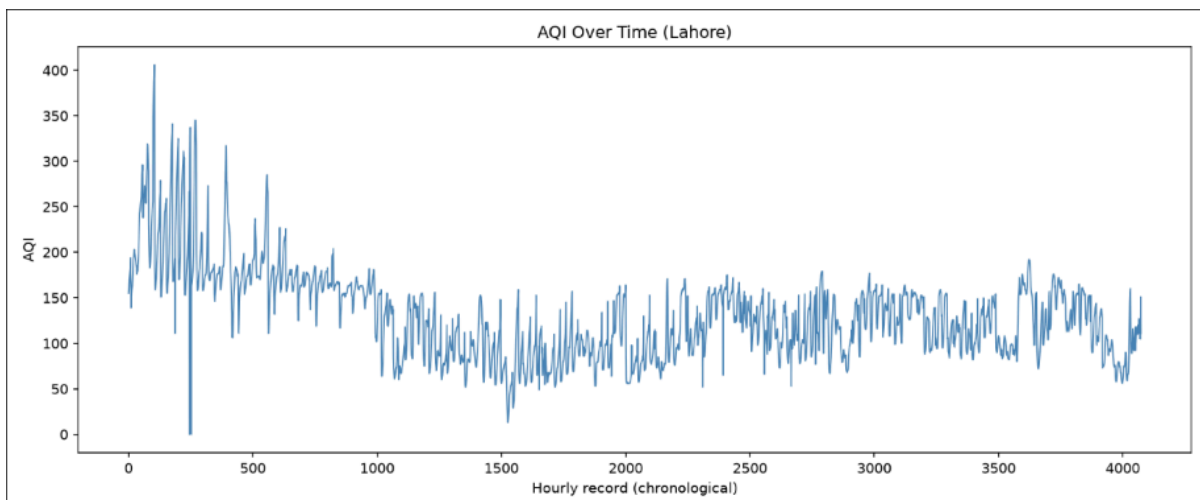


Figure 1: AQI over time (hourly records, chronological)

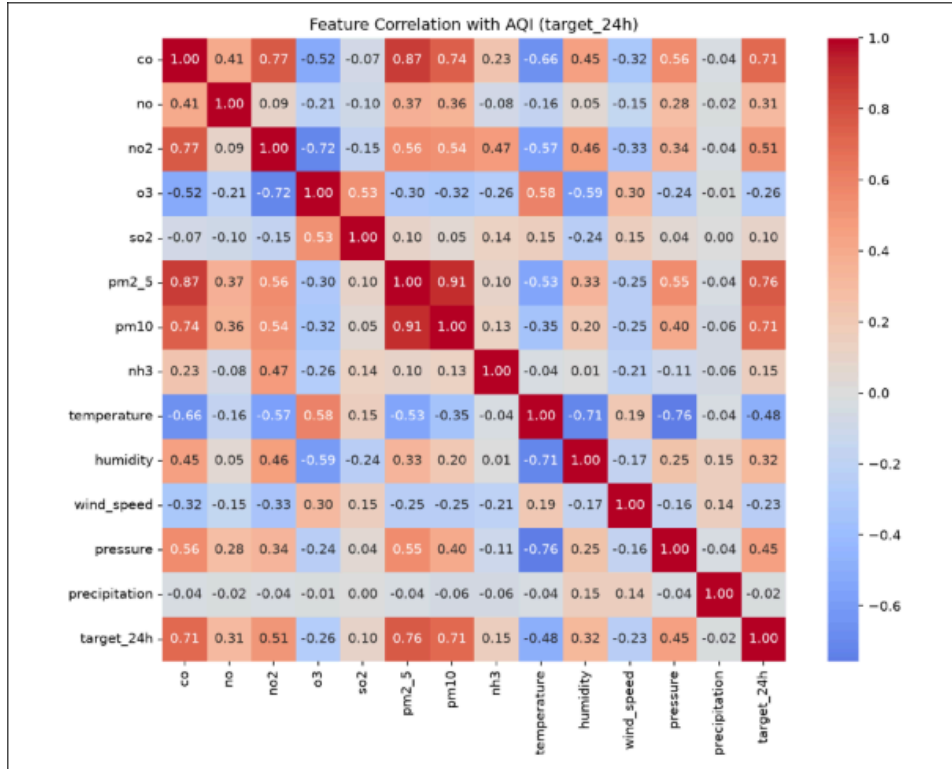


Figure 2: Correlation heatmap between pollutant/weather features and AQI

AQI shows a clear downward trend and stabilization over the collection window, starting noisy and high (peaking above 350-400) before settling into a steadier band roughly between 80-200 in more recent records. The correlation heatmap shows PM2.5 and PM10 are by far the strongest predictors of AQI (0.76 and 0.71 respectively), consistent with AQI being calculated directly from particulate concentrations; CO and NO2 also correlate positively and reasonably strongly (0.71 and 0.51). Temperature and pressure show the strongest negative/positive relationships among the weather variables (-0.48 and 0.45), suggesting weather conditions play a real but secondary role compared to the pollutant concentrations themselves.

4. Feature Engineering

The feature pipeline computes the following categories of features from the raw API data:

- Time-based features: hour of day, day, month, day of week
- Derived/rate features: PM2.5 change rate, AQI target change rate
- Rolling averages: PM2.5 rolling 3-hour and 24-hour averages
- Raw pollutant concentrations and weather variables

Lag features (previous-hour AQI/pollutant values as direct model inputs) were also tested during development, but were dropped from the final feature set: with roughly 58–169 days of historical data, adding lags reduced the amount of usable training data (rows require a full lag window to be valid) and introduced multicollinearity with the rolling-average features already in use, which hurt rather than helped generalization on this dataset size.

5. Feature Pipeline & Feature Store

Raw data is fetched, features are computed, and results are written to a Hopsworks Feature Group (aqi_lahore_features) using the Hopsworks free tier as the feature store. The pipeline runs automatically every hour via a scheduled GitHub Actions workflow (feature_pipeline.yml).

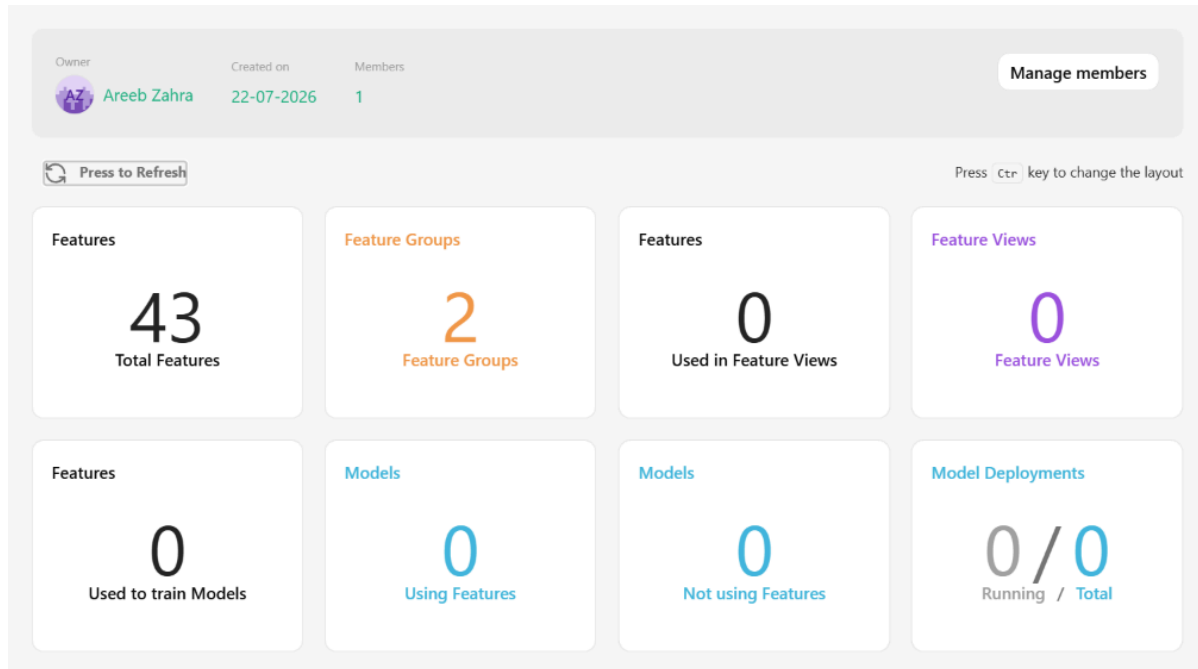


Figure 3: Hopsworks Feature Store overview — 2 feature groups, 43 total features

6. Historical Backfill

The feature pipeline script was run repeatedly across a range of past dates to reconstruct historical (features, target) pairs, producing the initial training dataset used before hourly automation took over.

7. Model Training & Evaluation

Three model families were trained and compared for each of the three forecast horizons: Ridge Regression, RandomForest, and a TensorFlow feed-forward neural network.

7.1 Classical Models (Ridge, RandomForest)

Ridge Regression and RandomForest were trained on an 80/20 chronological train/test split (no shuffling, to avoid leaking future information into training) and evaluated with RMSE, MAE, and R^2 .

7.2 TensorFlow Model

A small feed-forward neural network (2 hidden layers, L2 regularization and dropout) was trained per horizon. Two adjustments were necessary beyond the default setup:

- Target scaling: the AQI target was standardized (not just the input features) — without this, the network converged toward predicting near the mean, resulting in negative R^2 scores.
- Ensemble averaging: a single network's R^2 varied substantially between runs on this dataset size (~1,300 training rows) purely due to random weight initialization. Training 5 networks with different seeds per horizon and averaging their predictions reduced this variance and produced a more reliable estimate of true model performance.

7.3 Results Comparison

Horizon	Model	RMSE	MAE	R^2
---------	-------	------	-----	-------

24h	Ridge Regression	24.622	19.539	0.315
24h	TensorFlow (5-model ensemble)	26.019	21.425	0.236
48h	RandomForest	26.453	21.424	0.207
48h	TensorFlow (5-model ensemble)	30.414	25.427	-0.048
72h	RandomForest	28.228	23.063	0.093
72h	TensorFlow (5-model ensemble)	30.468	25.581	-0.058

The classical models outperformed the TensorFlow ensemble at all three horizons, and the gap widens as the horizon lengthens: Ridge beat TensorFlow by a moderate margin at 24h (R^2 0.315 vs 0.236), RandomForest's advantage grew at 48h (0.207 vs -0.048), and was largest at 72h (0.093 vs -0.058), where the TensorFlow ensemble performed worse than simply predicting the mean AQI. This is consistent with the dataset size available (roughly 1,300 training rows): tree-based and linear models have stronger inductive biases for small tabular datasets, while a neural network typically needs substantially more data to reliably learn useful structure rather than noise, especially as the prediction task grows harder at longer horizons. Ridge/RandomForest were therefore selected as the deployed models for live predictions in the dashboard, with the TensorFlow ensemble retained in the Model Registry for comparison.

8. Model Registry

The best model per horizon (by R^2) is registered to the Hopsworks Model Registry, along with its evaluation metrics and an input example. The TensorFlow ensembles are registered separately (aqi_model_24h_tf, etc.) alongside the classical models, so all approaches remain available for comparison.

Name	Latest version	Author	Deployed versions	Description	
aqi_tf_y_scalers	2			Per-horizon target StandardScalers -- TF model outputs are scaled an...	View all versions
aqi_tf_scaler	2			StandardScaler fitted on training features, required to use the Tensor...	View all versions
aqi_model_72h_tf	2			AQI forecast model for t+72h horizon (ensemble of small TensorFlow...	View all versions
aqi_model_48h_tf	2			AQI forecast model for t+48h horizon (ensemble of small TensorFlow...	View all versions
aqi_model_24h_tf	2			AQI forecast model for t+24h horizon (ensemble of small TensorFlow...	View all versions
aqi_model_72h	15			AQI forecast model for t+72h horizon (RandomForest), daily retrain o...	View all versions
aqi_model_48h	15			AQI forecast model for t+48h horizon (RandomForest), daily retrain o...	View all versions
aqi_model_24h	15			AQI forecast model for t+24h horizon (Ridge), daily retrain on polluta...	View all versions

Figure 5: Hopsworks Model Registry — Ridge/RandomForest (v15) and TensorFlow ensembles (v2), all registered

9. Automated CI/CD Pipeline

Two GitHub Actions workflows automate the system:

- Hourly feature pipeline: fetches new data and appends features to the Feature Store

- Daily training pipeline: retrains all models, including a dedicated TensorFlow step. Because TensorFlow's dependencies conflict with the Hopsworks client library (both pin incompatible versions of numpy, protobuf, and grpcio), the daily workflow is split into three sequential jobs — export data (Hopsworks env) → train TensorFlow model (isolated env) → register model (Hopsworks env) — passing data between jobs as workflow artifacts rather than sharing a single Python environment.

During development, the Hopsworks free-tier monthly budget was temporarily exhausted, which required pausing both workflows until the following billing cycle reset it. Both workflows are now enabled and running successfully on schedule, shown below.

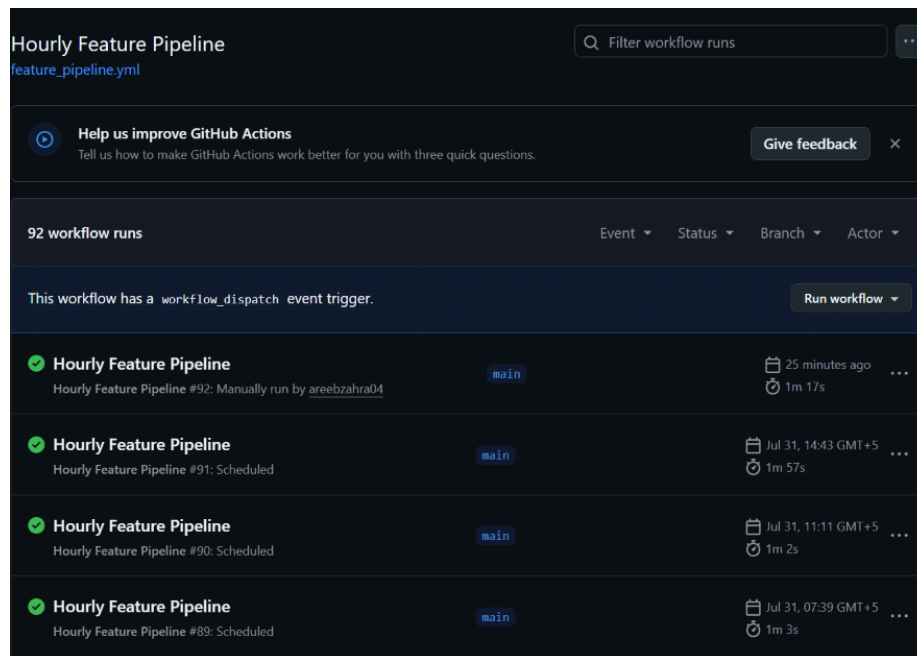


Figure 6: Hourly feature pipeline — scheduled runs, all green

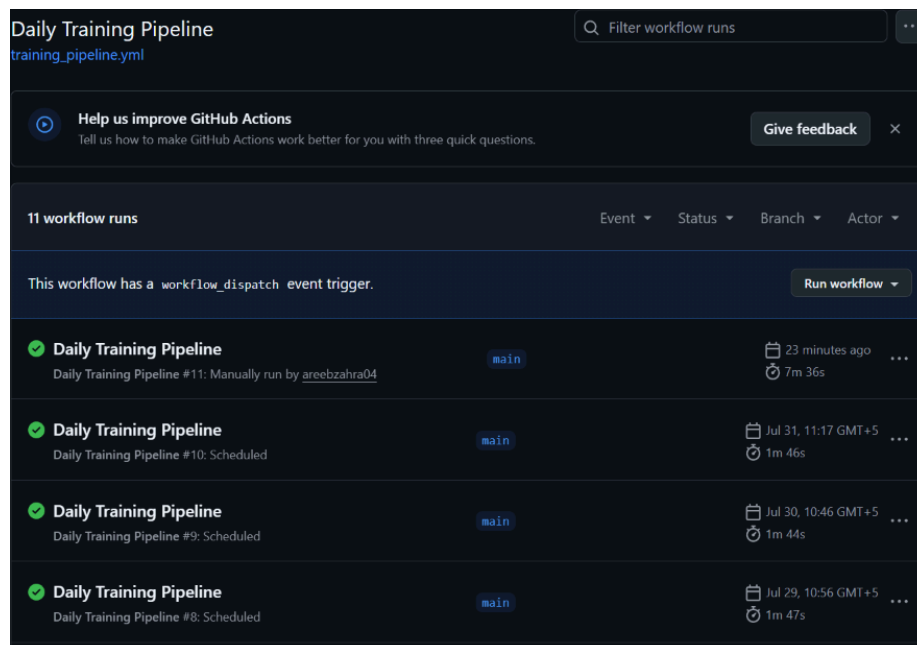


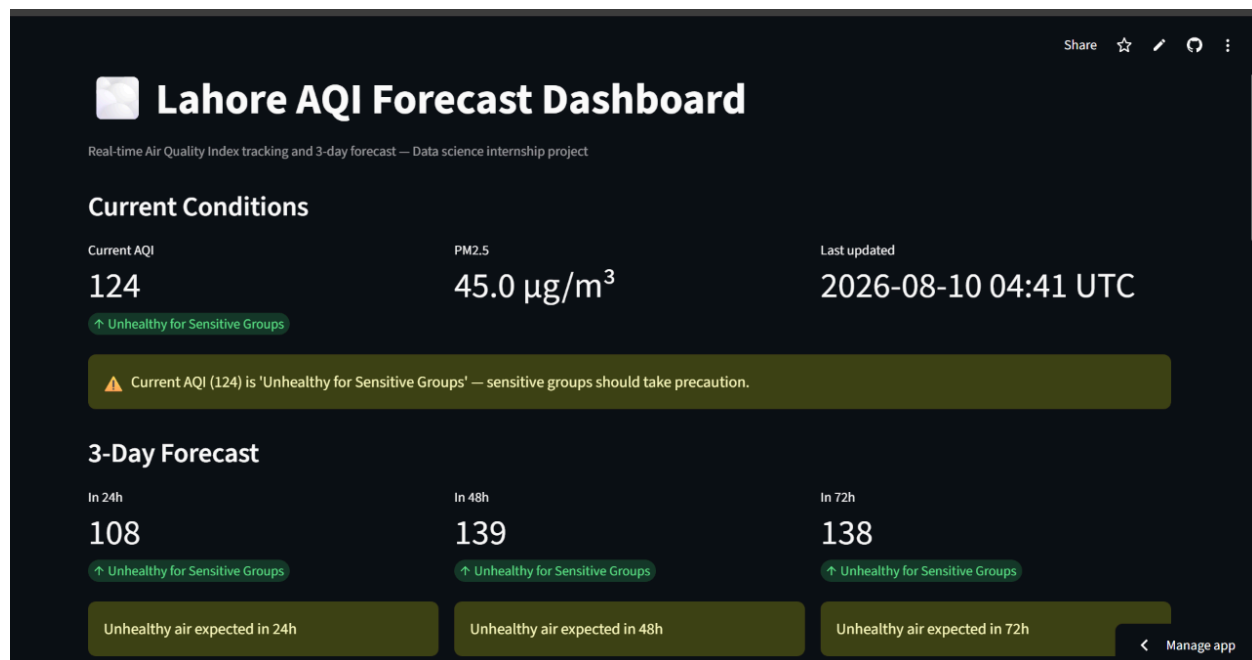
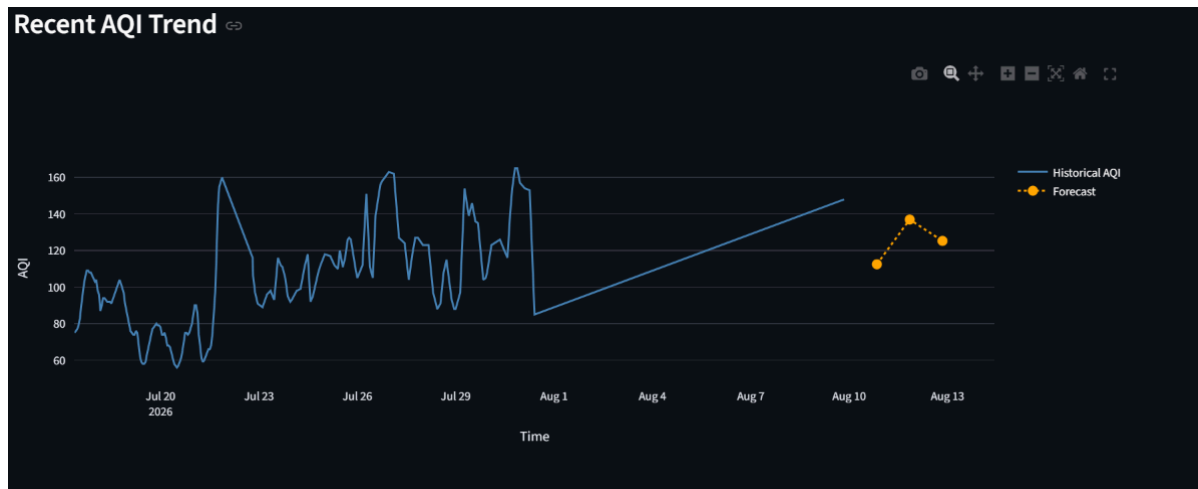
Figure 7: Daily training pipeline — scheduled runs, all green

10. Dashboard

The dashboard is built with Streamlit and displays:

- 3-day forecast cards (t+24h, t+48h, t+72h predicted AQI)
- A historical AQI trend chart
- SHAP-based feature importance, explaining which inputs drove each prediction
- Hazard-level alerts when forecasted AQI crosses unhealthy thresholds

The dashboard is deployed and publicly accessible at: <https://nepgz73usmnjmdfmbmt5wx.streamlit.app/>



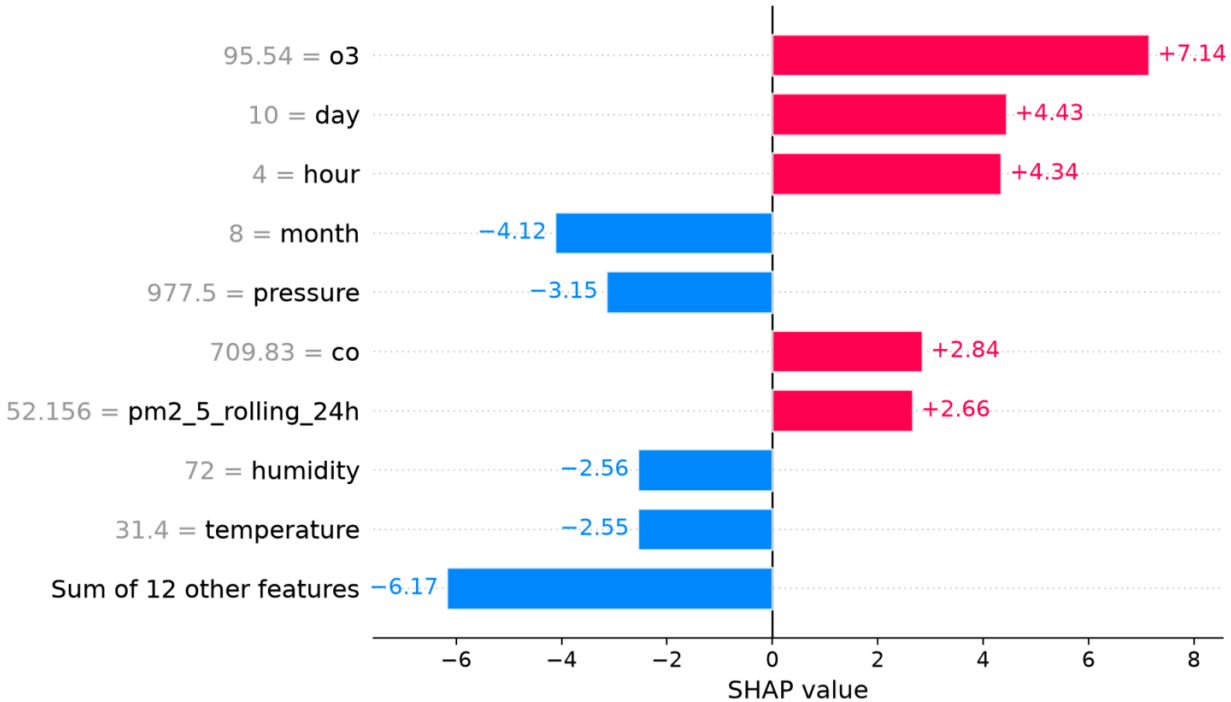


Figure 8: Dashboard — recent AQI trend with 3-day forecast overlay

11. Limitations & Future Work

- Training data is limited to a few months of history; a longer collection window would likely improve all models, especially the TensorFlow network
- A sequence model (LSTM/GRU) using recent AQI history as direct input, rather than flat rolling-average features, is a natural next step for the TensorFlow approach
- Free-tier resource limits (Hopsworks monthly budget) constrain how much historical backfill and how frequent retraining is sustainable long-term

12. Conclusion

This project delivered a complete, serverless AQI forecasting system for Lahore: automated hourly data collection into a Hopsworks Feature Store, a daily retraining pipeline comparing three model families (Ridge, RandomForest, and TensorFlow) across three forecast horizons, a Model Registry tracking all trained models, and a live, publicly deployed dashboard combining real-time conditions, 3-day forecasts, SHAP explanations, and hazard alerts. The key modeling takeaway is that classical models (Ridge and RandomForest) outperformed a TensorFlow neural network at every horizon on this dataset, with the gap widening substantially at longer horizons — a direct consequence of limited training data (roughly 1,300 hourly rows) rather than any inherent limitation of deep learning for this problem. With a longer collection window and a sequence-aware architecture (e.g. LSTM/GRU) rather than flat tabular features, TensorFlow would be the natural candidate to close or reverse this gap in future iterations.